

# 1. CNN Architecture Description

The model is a custom deep Convolutional Neural Network (CNN) featuring three sequential feature extraction blocks and a dense classifier.

- **Architecture Breakdown:**

- **Layer 1:** 32 filters ( $3 \times 3$ ), BatchNorm, ReLU, MaxPool.
- **Layer 2:** 64 filters ( $3 \times 3$ ), BatchNorm, ReLU, MaxPool.
- **Layer 3:** 128 filters ( $3 \times 3$ ), BatchNorm, ReLU, MaxPool.
- **Classifier:** Flattened vector  $\rightarrow$  512 Hidden Units  $\rightarrow$  Dropout (0.5)  $\rightarrow$  100 Output Classes.

- **Design Choices:**

- **Batch Normalization:** Integrated after every convolution to stabilize the internal covariate shift, allowing for a higher learning rate and faster convergence.
- **Increased Depth:** Three layers were chosen over two to capture the complex geometric nuances of 100 different sketch categories.
- **Dropout:** A 50% dropout rate was applied in the fully connected layer to enforce redundancy and prevent the model from memorizing training-specific noise.

## 2. Training Procedure

The training was designed to be efficient while maximizing the model's ability to generalize to unseen sketches.

- **Hyperparameters:**

- **Optimizer:** Adam (lr=0.001). Chosen for its adaptive learning rate properties which handle sparse gradients well.
- **Loss Function:** Cross-Entropy Loss, suitable for multi-class classification.
- **Batch Size:** 128. Balances memory efficiency with stable gradient estimates.
- **Scheduler:** ReduceLROnPlateau (patience=2). Automatically decays the learning rate when validation progress stalls.

- **Data Preprocessing & Split Strategy** Images were resized to  $28 \times 28$  and normalized to a range of  $[-1, 1]$ .

- **Augmentation:** RandomRotation( $15^\circ$ ) was applied only to the training set to simulate the natural variance in how humans draw.
- **Dataset Scale:** The dataset contains a total of **9,900 samples** distributed across 100 classes. This results in roughly 99 sketches per category. Given the relatively small number of samples per class, the data pipeline was optimized to prevent overfitting.

- **Split Logic:** An 80/20 split was utilized, providing 7,920 images for training and 1,980 images for testing.
- **Data Augmentation Necessity:** Because we have fewer than 100 images per class, **Random Rotation (15°)** became a critical design choice. It artificially expands the dataset by showing the model "new" versions of the existing sketches, which is essential for a high-accuracy result (96.9%) with a smaller dataset.
- **Batch Strategy:** A **batch size of 128** was maintained. With a training set of 7,920 images, the model processes approximately **62 batches per epoch**. This allows for frequent weight updates, which helps the model converge quickly despite the smaller total image count.

### 3. Results & Discussion

- **Final Performance:** The model achieved a final test accuracy of **96.9%**.
- **Convergence Observations:**
  - **Underfitting:** The model was able to drive training loss down very quickly, suggesting the 3-layer depth was sufficient for the task complexity.
  - **Overfitting:** Minimal. The gap between training and test accuracy remained narrow throughout the 10 epochs, indicating that the Dropout and Data Augmentation successfully prevented memorization.
- **Strengths:** The combination of BatchNorm and Adam allowed the model to reach peak accuracy in very few epochs. It is highly robust to tilted or off-center drawings.
- **Weaknesses:** The model may struggle with highly similar classes (e.g., distinguishing between a "cruise ship" and a "sailboat") due to the low resolution of  $28 \times 28$  pixels.
- **Improvements:** Implementing **Global Average Pooling** could reduce the number of parameters in the classifier, and increasing the input resolution to  $64 \times 64$  would allow the model to learn finer details. Although I have doubts about this!

### 4. Performance Plots

